# Assignment 6
# Visitor Pattern: Advanced File System

Name: Alikhan Smagul

Group: SE-2529

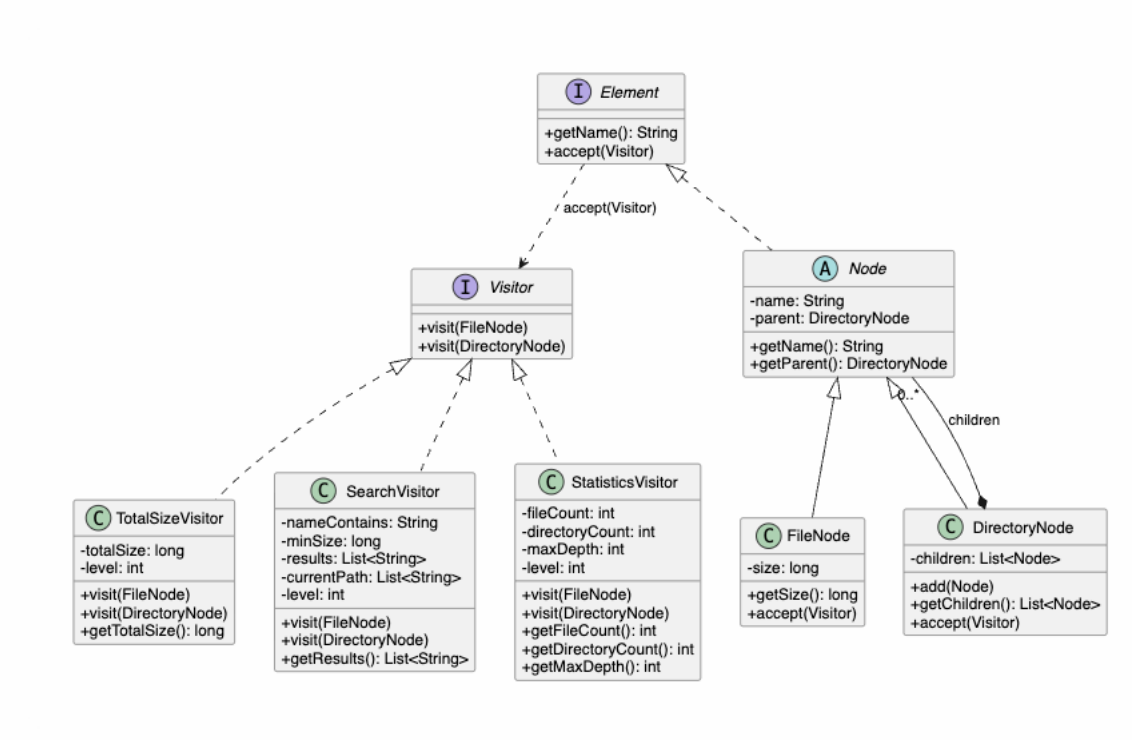
Repository: https://github.com/alihzzz/SDP_Assignment6.git

## 1. Purpose and design choices

The project is an in-memory hierarchical file system (Java 17, Maven) with three independent operations implemented through the Visitor pattern: total size, search and statistics. Elements only store data and accept visitors; all operations live in visitor classes, so a new operation does not change the elements.

- **Object model.** Element declares getName() and accept(Visitor). FileNode (name, non-negative size) and DirectoryNode (named children, any nesting) extend the abstract class Node, which holds the name and the parent link. Node exists so that the parent can be set only inside the package, without casts.
- **Double dispatch.** Step 1: element.accept(visitor) is chosen by the runtime type of the element. Step 2: inside it, visitor.visit(this) is resolved by the compiler from the static type of this (FileNode or DirectoryNode). No instanceof, getClass(), reflection or casts are used.
- **Tree integrity.** DirectoryNode.add() throws an exception for null, a child that already has a parent, a cycle (the child is the directory itself or one of its ancestors, found by walking getParent()) and a duplicate sibling name. FileNode rejects a negative size. A rejected change leaves the tree unchanged.
- **Traversal owner: the visitors.** visit(DirectoryNode) loops over the children and calls child.accept(this), so every reachable node is visited exactly once per operation. An empty directory just has no children to visit.
- **Reuse of visitors.** Visitors are reusable. The field level is 0 when no traversal is running; a visit that starts at level 0 resets the old result first. The result therefore does not depend on run order.
- **Depth and paths.** Depth is the value of level for the visited node: root = 0, its children = 1, and so on. SearchVisitor keeps currentPath (directory names below the root, the root name is not included); a file path is these names plus the file name joined with "/". getResults() returns a sorted copy (natural String order). If the visited root is a file, its path is its name. A file matches when its name contains the given text and size >= minSize (inclusive).

## 2. UML class diagram



## 3. Clean Code justifications

**a) Meaningful naming.** SearchVisitor(String nameContains, long minSize) and DirectoryNode.isSelfOrAncestorOf(Node). The names state the intent, so the code is readable without comments.

**b) Single Responsibility.** FileNode and DirectoryNode only hold data and dispatch; TotalSizeVisitor sums, SearchVisitor filters, StatisticsVisitor counts. Each class has one reason to change.

**c) Small methods.** SearchVisitor.matches(FileNode) and buildPath(String), StatisticsVisitor.updateMaxDepth() and reset(). Each method does one thing, so it is easy to test and explain.

**d) Encapsulation.** DirectoryNode.getChildren() returns Collections.unmodifiableList(children) and Node.setParent() is package-private. The tree can be changed only through add(), which checks all invariants.

**e) No type-checking chains.**

```
public void accept(Visitor visitor) { visitor.visit(this); }   // FileNode
```

Overload resolution replaces if (node instanceof FileNode) ... else ... chains, so adding a new operation never touches the elements.

## 4. Sample outputs

Demo tree: root contains photo.png (500), docs (report.txt 100, archive with old_report.txt 300, notes.txt 20), readme.txt (10) and an empty directory. Output of Main:

```
Total size: 930
Search (name contains 'report', size >= 150):
  docs/archive/old_report.txt
Files: 5
Directories: 4
Max depth: 3
```

Other search configurations on the same tree (threshold is inclusive, paths are sorted and root-relative):

```
search("report", >= 100) -> [docs/archive/old_report.txt, docs/report.txt]
search("report", >= 101) -> [docs/archive/old_report.txt]
search(".txt", >= 0)     -> [docs/archive/old_report.txt, docs/notes.txt,
                             docs/report.txt, readme.txt]
search("xyz", >= 0)      -> []
docs subtree, ".txt"     -> [archive/old_report.txt, notes.txt, report.txt]
empty directory stats    -> files=0, dirs=1, depth=0
```

## 5. Test summary

JUnit 5, run from the project root with: mvn test

| Test class | Tests | Checkpoints covered |
|---|---|---|
| DispatchTest | 2 | T1: both element types reach the correct visit overload |
| TotalSizeVisitorTest | 4 | T2 nested sums, T6 empty directory, T7 repeated run |
| SearchVisitorTest | 7 | T3 both filters, T4 sorted root-relative paths, T6 no match / empty, T7 repeated run |
| StatisticsVisitorTest | 3 | T5 counts and depth, T6 empty root, T7 repeated run |
| TreeIntegrityTest | 7 | T8 negative size, duplicate name, second parent, cycles rejected; valid nesting works |

Result: Tests run: 23, Failures: 0, Errors: 0, Skipped: 0 (BUILD SUCCESS).

## 6. Traversal and extension trade-offs

**Traversal.** Putting the loop inside the visitors keeps the elements very small and lets each visitor keep its own context (level, current path) naturally. The cost is that the same loop is repeated in all three visitors; a separate walker would remove this duplication, but it would have to pass depth and path information to the visitors.

**Extension.** StatisticsVisitor was added in a separate commit without changing FileNode, DirectoryNode or their accept methods: the extension boundary is the Visitor interface. Adding a new visitor is cheap. Adding a new element type is the weak point: a new visit method must be added to Visitor and implemented in every existing visitor, so the whole hierarchy changes.

## 7. GitHub link and sources

Repository: https://github.com/alihzzz/SDP_Assignment6.git